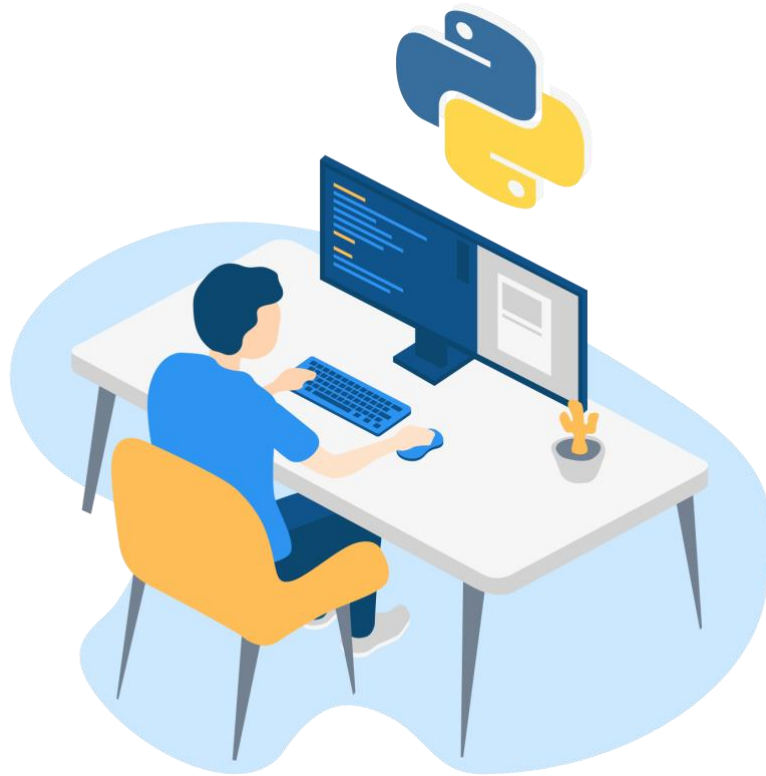




9.1 Tujuan

Setelah menyelesaikan bab ini dan mengikuti kegiatan praktikum, mahasiswa diharapkan mampu:

1. Memahami dan mengimplementasikan alur autentikasi dan otorisasi pengguna (login, logout, signup, dan proteksi view) menggunakan sistem bawaan Django.
2. Memahami dan menerapkan relasi basis data *One-to-Many* menggunakan `ForeignKey` untuk menghubungkan model baru (`Comment`) ke model yang sudah ada (`Post`).
3. Membuat dan memproses `ModelForm` untuk model yang berelasi (`CommentForm`) guna menangani input pengguna pada halaman detail postingan.

9.2 Pengantar

Pada Bab 8, kita telah belajar dalam hal keamanan dengan menyembunyikan tautan "Edit" dan "New Post" menggunakan `{% if user.is_authenticated %}`. Namun, keamanan tersebut baru sebatas tampilan (*client-side*). Pengguna yang pintar masih bisa menebak URL (misal: `/post/new/`) dan mengakses halaman tersebut.

Di bab ini, kita akan mengimplementasikan keamanan yang sesungguhnya di sisi server (*server-side*), mengatur alur *login* dan *logout*, serta memperkaya aplikasi kita dengan fitur komentar, yang akan memperkenalkan kita pada konsep relasi model.

9.2.1 Autentikasi vs Otorisasi

- **Autentikasi (Authentication)**
Proses verifikasi *siapa* user. Biasanya digunakan untuk mekanisme *login* dan *logout*. Django menyediakan sistem bawaan untuk memproses autentikasi.
- **Otorisasi (Authorization)**
Proses menentukan *apa yang boleh user lakukan* setelah Anda *login*. Contoh: "Hanya pengguna yang terautentikasi yang boleh membuat postingan baru."

9.2.2 Alur autentikasi bawaan Django

Django mempermudah kita dengan menyediakan *views* dan *URL patterns* bawaan untuk *login* dan *logout*. Kita hanya perlu mengkonfigurasi dan membuat *template*-nya.

- **URL:** Kita dapat mengimpor URL autentikasi bawaan (`django.contrib.auth.urls`) ke dalam file `urls.py` utama kita. Django akan secara otomatis menyediakan *path* seperti `/accounts/login/` dan `/accounts/logout/`.
- **Template:** Django akan mencari *template* di lokasi spesifik: `registration/login.html`. Kita yang harus membuat file ini.
- **Settings:** Kita perlu memberi tahu Django ke mana harus mengarahkan pengguna setelah mereka berhasil *login*. Konfigurasi ini diatur dalam `mysite/settings.py` menggunakan variabel `LOGIN_REDIRECT_URL`.

9.2.3 Otorisasi view dengan `@login_required`

Kita dapat menggunakan cara *server-side* untuk mengamankan *view* dengan menggunakan `@login_required`. `@login_required` adalah sebuah *decorator*, sebuah fungsi Python yang "membungkus" *view* kita.

```
1. # blog/views.py
2. from django.contrib.auth.decorators import login_required
3.
4. @login_required
5. def post_new(request):
```

```
6. # ... logika view ...
```

Jika pengguna yang belum *login* mencoba mengakses `post_new`, Django akan secara otomatis mengalihkannya ke halaman *login* (yang didefinisikan di `settings.py`).

9.2.4 Relasi Model

Bagaimana jika kita ingin menambahkan komentar pada suatu postingan?

Sebuah komentar harus "terikat" pada satu postingan. Dalam database hal ini disebut relasi *One-to-Many* (Satu Postingan, Banyak Komentar).

Di Django, kita dapat menggunakan `models.ForeignKey` untuk mengatasi masalah ini.

```
1. # blog/models.py
2. class Comment(models.Model):
3.     post = models.ForeignKey(Post, on_delete=models.CASCADE, related_name=
       'comments')
4.     author = models.CharField(max_length=200)
5.     text = models.TextField()
6.     created_date = models.DateTimeField(default=timezone.now)
7.
8.     def __str__(self):
9.         return self.text
```

- `post = models.ForeignKey(...)`: syntax ini adalah field kunci yang menghubungkan `Comment` ke `Post`.
- `on_delete=models.CASCADE`: syntax ini memberi tahu basis data bahwa jika sebuah `Post` dihapus, semua `Comment` yang terikat padanya juga harus ikut terhapus.
- `related_name='comments'`: syntax ini memungkinkan kita untuk mengakses semua komentar dari sebuah objek `Post`. Contoh: `post_object.comments.all()`.

9.2.5 Mengelola Form Komentar

Setelah model `Comment` ada, kita bisa membuat `ModelForm` untuknya (misal, `CommentForm`). Logika ini biasanya ditempatkan di dalam *view* `post_detail`, karena form komentar ditampilkan di halaman detail postingan.

Artinya, *view* `post_detail` akan memiliki tanggung jawab ganda:

1. Permintaan GET:

Mengambil objek `Post` dan menampilkannya, serta menampilkan form komentar yang kosong.

2. Permintaan POST:

Memproses data form komentar yang baru disubmit.

9.3 Kegiatan Praktikum

Langkah 1: Persiapan

1. Buka kembali folder django hasil praktikum pada Bab sebelumnya
2. Aktifkan *virtual environment* pada folder tersebut tersebut.

Terminal

```
myenv\Scripts\activate.bat
```

Langkah 2 : Membangun halaman login dan logout

1. Buka file `mysite/settings.py` dan tambahkan variabel ini di bagian paling bawah untuk menentukan halaman tujuan setelah login

`mysite/settings.py`

```
2. # mysite/settings.py
3. LOGIN_REDIRECT_URL = '/'
```

2. Buka file `mysite/urls.py` (file URL di *root* proyek) dan tambahkan baris warna kuning untuk autentikasi bawaan Django.

`mysite/urls.py`

```
1. from django.contrib import admin
2. from django.urls import path, include
3. from django.contrib.auth import views
4. from blog import views as blog_views
5.
6. urlpatterns = [
7.     path('admin/', admin.site.urls),
8.     path('accounts/login/', views.LoginView.as_view(), name='login'),
9.     path('accounts/logout/', blog_views.custom_logout, name='logout'),
10.    path('', include('blog.urls')),
11.]
```

4. Buka file `blog/views.py` kemudian tambahkan baris warna kuning untuk menghandle request logout.

blog/views.py

```
1. from django.shortcuts import render, get_object_or_404, redirect
2. from django.utils import timezone
3. from .models import Post
4. from .forms import PostForm
5. from django.contrib.auth import logout
6. from django.shortcuts import redirect
7.
8. #kode sebelumnya
9.
10. def custom_logout(request):
11.     logout(request)
12.     return redirect('/')

```

5. Buat folder bernama `registration`. di dalam `blog/templates/`. Buat file baru bernama `login.html`.

blog/templates/login.html

```
1. {% extends "blog/base.html" %}
2.
3. {% block content %}
4.     {% if form.errors %}
5.         <p>Your username and password didn't match. Please try again.</p>
6.     {% endif %}
7.
8.     <h2>Please login:</h2>
9.     <form method="POST" action="{% url 'login' %}">
10.         {% csrf_token %}
11.         {{ form.as_p }}
12.         <button type="submit" class="btn btn-secondary">Login</button>
13.     </form>
14. {% endblock %}

```

6. Buka `blog/templates/blog/base.html`. Ganti tautan "New Post" dengan blok `login/logout` yang lebih lengkap.

Cari dan hapus

blog/templates/blog/base.html.

```
1. {% if user.is_authenticated %}
2.     <a href="{% url 'post_new' %}" class="top-menu">+ New Post</a>
3. {% endif %}

```

Ganti dengan:

```
1. {% if user.is_authenticated %}

```

```

2. <a href="{% url 'post_new' %}" class="top-menu">+ New</a>
3. <p class="top-menu">Hello {{ user.username }} <small>(<a href="{% url 'logout' %}">Log o
   ut</a></small></p>
4. {% else %}
5. <a href="{% url 'login' %}" class="top-menu">Login</a>
6. {% endif %}

```

7. Jalankan server dan buka browser pada alamat <http://127.0.0.1:8000/>. Klik menu login dan masuk dengan menggunakan akun admin yang sudah disetting sebelumnya pada modul 6.

Langkah 3: Melindungi Views (Otorisasi Server-Side)

1. Buka `blog/views.py`. Impor *decorator* `@login_required` di bagian atas.

`blog/views.py`.

1. `from django.contrib.auth.decorators import login_required`

2. Tambahkan *decorator* `@login_required` tepat di atas fungsi *view* `post_new` dan `post_edit`.

`blog/views.py`

```

1. # blog/views.py
2. @login_required
3. def post_new(request):
4. # ... (sisia fungsi tetap sama) ...
5.
6. @login_required
7. def post_edit(request, pk):
8. # ... (sisia fungsi tetap sama) ...

```

3. Sekarang, coba akses halaman *new post* secara manual dengan mengetik `http://127.0.0.1:8000/post/new/` , Sekarang kita tidak akan bisa mengakses halaman tersebut. Halaman akan otomatis diarahkan ke halaman *login*.

Langkah 4: Membuat Model Comment

1. Buka `blog/models.py`. Tambahkan model `Comment` di bawah model `Post` dengan definisi berikut:

`blog/models.py`

```
1. #kode sebelumnya
2.
3. class Comment(models.Model):
4.     post = models.ForeignKey('blog.Post', on_delete=models.CASCADE
5.     , related_name='comments')
6.     author = models.CharField(max_length=200)
7.     text = models.TextField()
8.     created_date = models.DateTimeField(default=timezone.now)
9.     approved_comment = models.BooleanField(default=False)
10.
11.     def approve(self):
12.         self.approved_comment = True
13.         self.save()
14.
15.     def __str__(self):
16.         return self.text
```

2. Buka `blog/admin.py`. Tambahkan kode berikut

`blog/admin.py`.

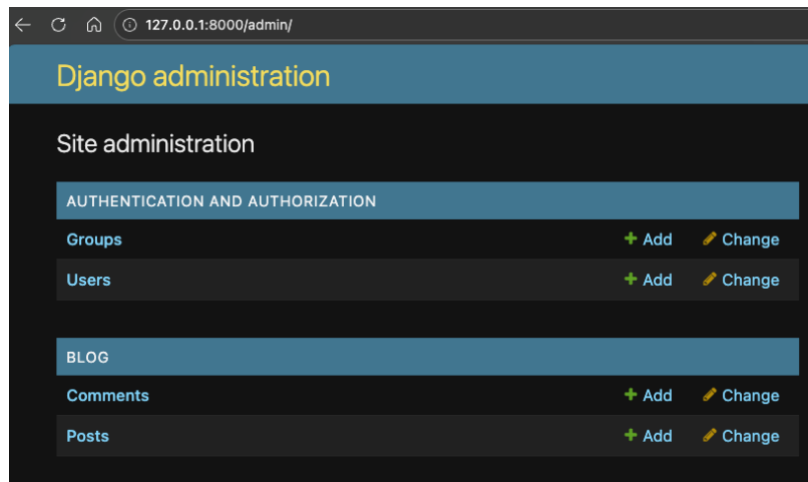
```
1. from django.contrib import admin
2. from .models import Post, Comment
3.
4. admin.site.register(Post)
5. admin.site.register(Comment)
```

3. Hentikan server dan jalankan migrasi untuk model `Comment` yang baru.

Terminal

```
1. python manage.py makemigrations blog
2. python manage.py migrate blog
```

4. Jalankan server kemudian akses halaman <http://127.0.0.1:8000/admin/>, kita akan melihat tampilan Comments pada Blog.



5. Buka file `blog/templates/blog/post_detail.html` tambahkan baris berikut sebelum tag `{% endblock %}`

`blog/templates/blog/post_detail.html`

```
1. <hr>
2. {% for comment in post.comments.all %}
3.     <div class="comment">
4.         <div class="date">{{ comment.created_date }}</div>
5.         <strong>{{ comment.author }}</strong>
6.         <p>{{ comment.text|linebreaks }}</p>
7.     </div>
8. {% empty %}
9.     <p>No comments here yet :(</p>
10. {% endfor %}
```

6. Kita juga dapat memberi tahu pengunjung tentang komentar pada halaman daftar postingan. Buka file `blog/templates/blog/post_list.html` dan tambahkan baris warna kuning berikut :

`blog/templates/blog/post_list.html`

```
1. {% extends 'blog/base.html' %}
2.
3. {% block content %}
4.     {% for post in posts %}
5.         <article class="post">
6.             <h2><a href="{% url 'post_detail' pk=post.pk %}">{{ post.title }}</a></h2>
7.             <p>Published: {{ post.published_date }}</p>
8.             <p>{{ post.text|linebreaksbr }}</p>
9.             <a href="{% url 'post_detail' pk=post.pk %}">Comments: {{ post
               .comments.count }}</a>
10.         </article>
```



```
11.     {% endfor %}  
12. {% endblock %}
```

7. Buat file `blog/forms.py` tambahkan baris berikut

`blog/forms.py`

```
1. from django import forms  
2. from .models import Post, Comment  
3.  
4. class PostForm(forms.ModelForm):  
5.     class Meta:  
6.         model = Post  
7.         fields = ('title', 'text',)  
8.  
9. class CommentForm(forms.ModelForm):  
10.    class Meta:  
11.        model = Comment  
12.        fields = ('author', 'text',)
```

8. Buka file `blog/templates/blog/post_detail.html` tambahkan baris berikut sebelum kode `{% for comment in post.comments.all %}` :

`blog/templates/blog/post_detail.html`

```
1. <a class="btn btn-  
    secondary" role="button" href="{% url 'add_comment_to_post' pk=post.pk %}"  
    >Add comment</a>
```

9. Buka file `blog/urls.py` tambahkan baris warna kuning ke `urlpatterns`:

```
1. from django.urls import path  
2. from . import views  
3.  
4. urlpatterns = [  
5.     path('', views.post_list, name='post_list'),  
6.     path('post/<int:pk>/', views.post_detail, name='post_detail'),  
7.     path('post/new/', views.post_new, name='post_new'),  
8.     path('post/<int:pk>/edit/', views.post_edit, name='post_edit'),  
9.     path('post/<int:pk>/comment/', views.add_comment_to_post, name='add_co  
    mment_to_post'),  
10.  
11.]
```

10. Buka `blog/views.py` dan tambahkan baris berikut :

`blog/views.py`

```
1. #kode sebelumnya  
2.
```

```

3. def add_comment_to_post(request, pk):
4.     post = get_object_or_404(Post, pk=pk)
5.     if request.method == "POST":
6.         form = CommentForm(request.POST)
7.         if form.is_valid():
8.             comment = form.save(commit=False)
9.             comment.post = post
10.            comment.save()
11.            return redirect('post_detail', pk=post.pk)
12.     else:
13.         form = CommentForm()
14.     return render(request, 'blog/add_comment_to_post.html', {'form': form})

```

11. Jangan lupa untuk mengimport `CommentForm` pada awal baris kode

`blog/views.py`

```

1. from .forms import PostForm, CommentForm

```

12. Buat file baru bernama `add_comment_to_post.html` pada folder `blog/templates/blog/`.
 Tambahkan kode berikut :

`blog/templates/blog/add_comment_to_post.html`

```

1. {% extends 'blog/base.html' %}
2.
3. {% block content %}
4. <h1>New comment</h1>
5. <form method="POST" class="post-form">{% csrf_token %}
6.     {{ form.as_p }}
7.     <button type="submit" class="save btn btn-secondary">Send</button>
8. </form>
9. {% endblock %}

```

Jalankan server dan buka browser pada alamat <http://127.0.0.1:8000/>. Klik salah satu postingan dan tambahkan komentar pada postingan tersebut



9.4 Rangkuman

- Django menyediakan alur autentikasi bawaan melalui `django.contrib.auth.urls` yang dapat kita gunakan dengan membuat *template* `registration/login.html`.
- Keamanan *server-side* diimplementasikan menggunakan *decorator* `@login_required` pada *views* yang perlu dilindungi.
- Relasi basis data *One-to-Many* diimplementasikan menggunakan `models.ForeignKey`, yang memungkinkan model `Comment` terikat pada model `Post`.
- Atribut `related_name` pada `ForeignKey` (misal, `related_name='comments'`) memungkinkan kita mengakses objek terkait dari *instance* model (misal, `post.comments.all()`).
- Sebuah *view* (seperti `post_detail`) dapat menangani logika ganda: menampilkan data (GET) dan memproses form (POST).

9.5 Tugas

Kerjakan tugas-tugas di bawah ini untuk memperkuat pemahaman dan keterampilan Anda.

1. Apa perbedaan fungsional antara `on_delete=models.CASCADE` dan `on_delete=models.SET_NULL` pada `ForeignKey`?
2. Jelaskan mengapa menggunakan `@login_required` di *view* jauh lebih aman daripada hanya menggunakan `{% if user.is_authenticated %}` di *template*.